INTERNSHIP PROJECT REPORT

Multi-Vendor E-Commerce Platform

A microservices-based marketplace with vendor dashboard, product filtering, cart and checkout, order tracking and PDF invoices

Submitted by	SINCHANA RAVI GOWDA
Internship	SPARKIIT - Full-Stack Development
Internship period	5 October 2026 - 20 November 2026
Technologies	Node.js, Express, SQLite, JWT, PDFKit, HTML/CSS/JavaScript
Source code	https://github.com/SinchanaRG26/sparkit_internship/tree/main/project3-multivendor-ecommerce
Report date	11 November 2026

Contents

- 1. Abstract and Introduction
- 2. Objectives
- 3. Methodology / Implementation
- 4. Source Code and Technical Details
- 5. Output Screenshots
- 6. Results and Analysis
- 7. Conclusion
- 8. References

1. Abstract and Introduction

Abstract

ShopSphere is a multi-vendor online store that I designed and built during my SPARKIIT internship. Several independent sellers list their products on one site, and a customer can put items from different sellers into a single cart, pay once, and then watch each item travel from the seller to the doorstep. Sellers get their own dashboard where they add products, see what has sold and mark orders as confirmed, packed, shipped or delivered. After an order is placed, the customer can download a PDF invoice.

Instead of writing one big program, I split the back end into three small services (users, products, orders) with a gateway in front of them. Each service runs as its own Node.js process and keeps its own SQLite database. The report explains how the pieces fit together, shows the working application through screenshots of the customer and of all four demo vendors, and records the test results: 80 automated end-to-end checks, all passing.

Introduction

When I read the brief (vendor dashboard, product listing with filtering, cart with checkout, order tracking with invoice, using microservices and cart logic) I wanted to avoid building a shop that merely looks right. The interesting problems in e-commerce are hidden: two people buying the last item at the same moment, prices that must not be trusted from the browser, an order that contains goods from three different sellers who each ship on their own schedule. I decided to make those the centre of the project and let the screens follow from them.

A **monolith** keeps every feature in one program and one database. With **microservices** each business area is a separate service that owns its data and talks to others over the network. The benefit is that a problem in one area stays there and each part can be changed or scaled on its own; the cost is that anything spanning two services, like checkout, needs extra care. I wanted to feel that trade-off first-hand, which is why the project is built this way.

The two concepts named in the brief appear in the project as follows:

- **Microservices:** user, catalog and order services plus a gateway, one database per service, a token check in every service, and a private key-protected channel for service-to-service calls.
- **Cart logic:** a server-side cart that checks stock, applies coupons, adds GST and shipping, and a checkout that reserves stock, recalculates the price on the server and only then creates the order.

2. Objectives

The objectives, taken from the internship brief, and how each is met:

#	Objective	How it is met
1	Vendor dashboard	Overview with revenue, orders, units sold, a 7-day revenue chart and top products; product management (add, edit, delete); order fulfilment with status buttons.
2	Product listing and filtering	Search, category, vendor, price range, in-stock only, five sort orders and pagination; category and vendor counts update with the filters.
3	Cart and checkout system	Cart with quantity control and stock validation; coupons WELCOME10, FLAT100 and FREESHIP; 18% GST; shipping rule; address validation; COD, test card and test UPI payments.
4	Order tracking and invoice	Per-item status flow Placed, Confirmed, Packed, Shipped, Delivered (or Cancelled), an activity log, customer and vendor cancellation, and a downloadable PDF tax invoice.
5	Microservices architecture	User, catalog and order services with separate databases, an API gateway, JWT authentication and an internal-key protected service API.
6	Production-level quality	Hashed passwords, login throttling, rate limiting, input validation, role-based access, atomic stock handling, health checks, structured logs, restart-on-crash launcher and 80 automated tests.

3. Methodology / Implementation

3.1 System architecture

The browser talks only to the **API gateway** (port 4000). The gateway serves the storefront files, forwards each API route to the right service, adds a request id for tracing, applies a rate limit of 300 requests per minute per IP address and reports the combined health of all services. The services listen only on the local interface (127.0.0.1), so they cannot be reached from outside except through the gateway.

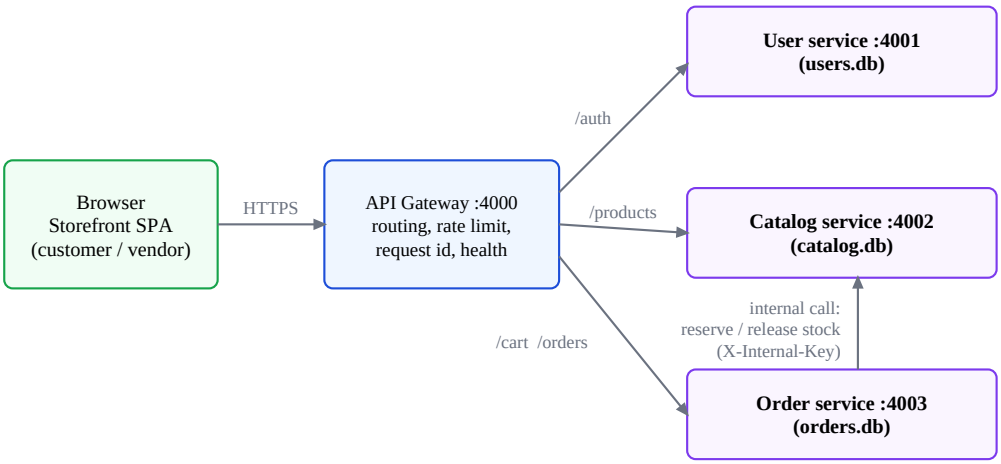


Figure 1: Architecture of the multi-vendor platform (gateway, three services, one database per service)

Component	Port	Responsibility	Database
API gateway	4000	Static storefront, routing, rate limiting, request ids, /api/health	-
user-service	4001	Registration, login, profile, list of vendors; roles customer and vendor	users.db
catalog-service	4002	Products, categories, search and filters, vendor product CRUD, atomic stock reserve and release	catalog.db
order-service	4003	Cart, coupons, pricing, checkout, orders, status flow, vendor orders and statistics, PDF invoice	orders.db

3.2 Security design

- **Authentication.** A successful login returns a JWT (HS256, 8 hours) containing the user id, name, role and shop name. Every service verifies the token itself; none trusts the gateway blindly.
- **Authorisation.** Role checks separate customers and vendors, and ownership checks make sure a vendor can edit only their own products and a customer can read only their own orders.
- **Passwords.** Stored as salted scrypt hashes (8 to 100 characters accepted); repeated failed logins are throttled.
- **Service-to-service calls.** The order service calls the catalog service on `/internal/*` routes protected by a shared key (compared in constant time). The gateway does not forward these routes and drops unknown headers.
- **Input safety.** All inputs are validated and trimmed, database queries are parameterised (a SQL-injection style search string is tested and is harmless) and all user text is HTML-escaped in the browser.

3.3 Cart and checkout logic

- **Cart.** Stored on the server per customer. Adding an item checks the available stock (including what is already in the cart) and a maximum quantity per product; the cart view marks items that have become unavailable.
- **Pricing rules.** Money is kept in integer paise. Subtotal, then coupon discount, then **18% GST on the discounted amount**, plus shipping of Rs. 49 that is free when the discounted amount is Rs. 999 or more. Coupons: WELCOME10 (10% off), FLAT100 (Rs. 100 off above Rs. 500), FREESHIP (free shipping).
- **Checkout.** (1) validate the address and payment method, (2) ask the catalog service to **reserve** stock atomically, (3) price the order again from trusted server-side prices, (4) save the order, its items and the first tracking event, (5) clear the cart. If any step fails, the reserved stock is released.
- **No overselling.** Reservation runs in a `BEGIN IMMEDIATE` SQLite transaction. When two customers want the last unit, one succeeds and the other receives HTTP 409; stock never goes negative.

3.4 Order tracking and invoice

An order can contain items from several vendors, so tracking is kept **per item**. Each vendor moves only their own items through PLACED, CONFIRMED, PACKED, SHIPPED and DELIVERED, and steps cannot be skipped. The order status is that of the slowest active item. An item can be cancelled by the customer or the vendor while it is PLACED or CONFIRMED; the stock is then returned to the catalog. Cash-on-delivery orders become PAID when delivered. Every change is written to an events table that the customer sees as the activity timeline. The invoice is generated on demand as a PDF with PDFKit and shows invoice number, addresses, items by vendor, discount, shipping, GST and total.

3.5 Technology stack

Layer	Technology	Purpose
Runtime	Node.js 22+	Server-side JavaScript, built-in fetch and node:sqlite
Services and gateway	Express 5	REST APIs, routing and static hosting
Databases	SQLite (WAL mode)	One embedded database per service, foreign keys and transactions
Authentication	jsonwebtoken, crypto.scrypt	JWT tokens and password hashing
Invoices	PDFKit	PDF tax invoice generation
Frontend	HTML, CSS, JavaScript (no framework)	Hash-routed single-page storefront, responsive, light and dark theme, SVG chart
Testing	Node scripts (fetch)	80 end-to-end checks against live services on temporary ports

3.6 Main API routes (through the gateway)

Method	Route	Who	Purpose
POST	/api/auth/register, /api/auth/login	public	Create account (customer or vendor with shop name) and sign in
GET	/api/products (query: q, category, vendor, minPrice, maxPrice, inStock, sort, page)	public	Search, filter, sort and paginate; returns category and vendor facets
GET	/api/products/:id	public	Product detail
POST, PUT, DELETE	/api/products	vendor	Create, edit and soft-delete own products
GET, POST, PUT, DELETE	/api/cart, /api/cart/items	customer	View cart (with coupon preview), add, change quantity, remove
POST	/api/checkout	customer	Place the order
GET	/api/orders, /api/orders/:id	customer	Order list and details with tracking events
POST	/api/orders/:id/cancel	customer	Cancel unshipped items
GET	/api/orders/:id/invoice	customer	Download PDF invoice
GET	/api/vendor/orders, /api/vendor/stats	vendor	Items to fulfil; sales statistics
PUT	/api/vendor/items/:id/status	vendor	Advance or cancel an order item
GET	/api/health	public	Gateway and service status

3.7 Frontend

The storefront is a single-page application written in plain JavaScript (no build step). It uses hash routes (shop, product, cart, checkout, orders, vendor dashboard), keeps the JWT in the browser's local storage, shows toasts for feedback, adapts to phones and tablets and follows the user's light or dark system theme. Product pictures are small vector illustrations chosen by product type, and a vendor can also supply a photo URL.

3.8 Running the project

`npm install` then `npm start`. The launcher starts the four processes, restarts any that crash, waits until all are healthy and loads demo data (4 vendors, 20 products, 1 customer and 3 sample orders) on the first run. The storefront is then available at http://localhost:4000.

3.9 Design decisions and the reasons behind them

Decision	What I could have done	Why I chose this
Database per service	One shared database	A shared database would make the services a monolith in disguise; separate files force every cross-service need to go through an API.
Integer paise for money	Floating-point rupees	0.1 + 0.2 is not 0.3 in floating point. With whole paise every total is exact and the invoice always matches the screen.
Reserve stock before saving the order	Reduce stock after the order is saved	Reserving first means a failed order can simply release the stock. The other way round could leave an order with no stock behind it.
Tracking per order item	One status per order	Items in one order come from different vendors who ship separately, so a single status would be wrong most of the time.
Soft-delete products	Remove the row	Old orders must still show what was bought, so a deleted product is only hidden from the shop.
Server-side pricing	Trust the totals sent by the browser	Anyone can edit a request. The browser sends only product ids and quantities.
Plain JavaScript front end	React or Angular	No build step, so the project runs after one npm install and I understand every line of it.

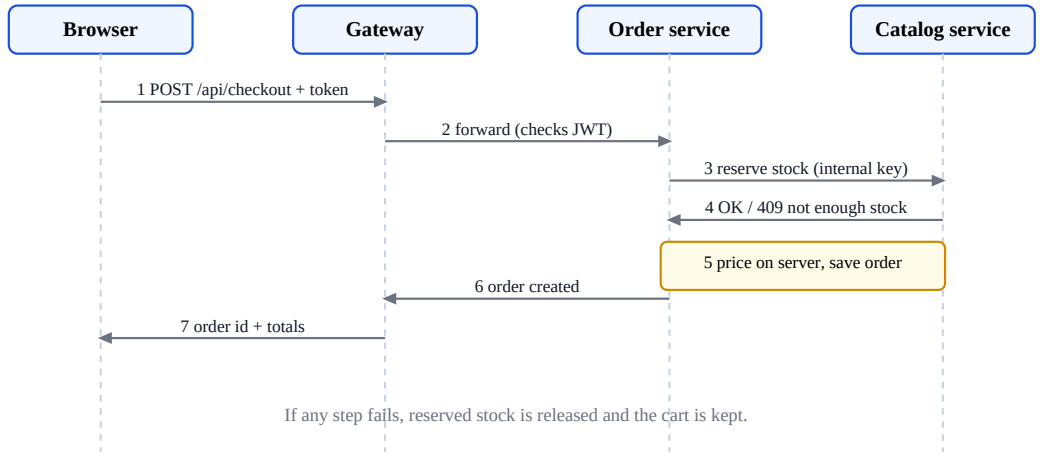


Figure 2: What happens when the customer presses Place order

3.10 Problems I ran into and how I solved them

- **Opening the page with Live Server.** My first run showed 'Request failed (404)' because I had opened index.html through VS Code Live Server. The page only loads data from the gateway, so it must be opened at localhost:4000 after npm start. I added this to the README.
- **The rupee sign in the PDF.** The standard fonts inside PDFKit do not contain the symbol, so the invoice printed a broken character. I used 'Rs.' in the PDF and kept the symbol on screen.

- **Last item, two buyers.** My first idea (read the stock, then subtract) can oversell when two requests arrive together. I moved the check and the subtraction into one SQLite transaction and wrote a test where two customers race for the last lamps.
- **Product pictures.** I started with emoji placeholders, which looked unprofessional, and replaced them with simple vector drawings of each product type, because I could not use copyrighted stock photos. A vendor can still paste a real photo link.
- **Order status for mixed orders.** When one vendor had shipped and another had not, the order needed a single sensible status. I defined it as the status of the slowest active item.

4. Source Code and Technical Details

The complete source code is in the GitHub repository: https://github.com/SinchanaRG26/sparkit_internship/tree/main/project3-multivendor-ecommerce. The main files are listed below; the front-end files (app.js, art.js, styles.css, index.html), the demo-data script seed.js and the test suite test/e2e.js are in the repository.

File	Role
`start-all.js`	Starts, monitors and restarts all services; loads demo data
`gateway/index.js`	API gateway
`services/user-service/index.js`	Accounts and login
`services/catalog-service/index.js`	Products, filters, atomic stock
`services/order-service/index.js`	Cart, checkout, orders, vendor tools
`services/order-service/pricing.js`	Discount, GST and shipping calculation
`services/order-service/invoice.js`	PDF invoice
`shared/auth.js, db.js, util.js`	JWT helpers, database helper and transaction wrapper, logging and validation
`gateway/public/*`	Storefront (HTML, CSS, JavaScript)
`seed.js, test/e2e.js`	Demo data and 80 end-to-end tests

4.1 start-all.js

```
/**
 * Starts every microservice as its own process, waits until they are healthy,
 * loads demo data on first run, and restarts a service if it crashes.
 *
 *   npm start
 */
const { spawn } = require('child_process');
const path = require('path');

const GATEWAY_PORT = +process.env.PORT || 4000;
const SERVICES = [
  { name: 'user-service', file: 'services/user-service/index.js', port: 4001, color: 36 },
  { name: 'catalog-service', file: 'services/catalog-service/index.js', port: 4002, color: 35 },
  { name: 'order-service', file: 'services/order-service/index.js', port: 4003, color: 33 },
```

```
  { name: 'gateway', file: 'gateway/index.js', port: GATEWAY_PORT, color: 32 },
];
const procs = new Map();
let stopping = false;

function launch(s) {
  const child = spawn(process.execPath, ['--no-warnings', path.join(__dirname, s.file)], {
    env: { ...process.env, PORT: String(s.port) }, stdio: ['ignore', 'pipe', 'pipe'],
  });
  const pipe = stream => stream.on('data', d => String(d).split('\n').filter(Boolean).forEach(l =>
      console.log(`\x1b[${s.color}m${s.name.padEnd(16)}\x1b[0m ${l}`)));
  pipe(child.stdout); pipe(child.stderr);
  child.on('exit', code => {
    if (stopping) return;
    console.log(`\x1b[31m${s.name} stopped (code ${code}); restarting in 2s\x1b[0m`);
    setTimeout(() => launch(s), 2000);
  });
  procs.set(s.name, child);
}

async function waitHealthy(timeoutMs = 20000) {
  const end = Date.now() + timeoutMs;
  while (Date.now() < end) {
    try {
      const r = await fetch(`http://127.0.0.1:${GATEWAY_PORT}/api/health`, { signal:
          AbortSignal.timeout(1500) });
      if ((await r.json()).ok) return true;
    } catch { /* not up yet */ }
    await new Promise(r => setTimeout(r, 400));
  }
  return false;
}

function shutdown() {
  stopping = true;
  for (const c of procs.values()) c.kill();
  setTimeout(() => process.exit(0), 300);
}
process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);

SERVICES.forEach(launch);
(async () => {
  if (!(await waitHealthy())) { console.log('Services did not become healthy in time. Check the logs
      above.'); return; }
  await new Promise(resolve => {
    const seed = spawn(process.execPath, ['--no-warnings', path.join(__dirname, 'seed.js')], { env: {
        ...process.env, BASE_URL: `http://127.0.0.1:${GATEWAY_PORT}` }, stdio: 'inherit' });
    seed.on('exit', resolve);
  });
  console.log(`\n  Storefront ready:  http://localhost:${GATEWAY_PORT}\n  Demo logins are listed on the Sign
      in page.\n  Press Ctrl+C to stop all services.\n`);
})();
```

4.2 shared/auth.js

```
/**
 * JWT helpers. Every service verifies tokens itself (stateless auth), and
 * service-to-service calls use a shared internal key that the gateway never forwards.
 */
const jwt = require('jsonwebtoken');
const crypto = require('crypto');
const { HttpError } = require('./util');

const SECRET = process.env.JWT_SECRET || 'dev-only-secret-change-me';
const INTERNAL_KEY = process.env.INTERNAL_KEY || 'dev-internal-key-change-me';

function sign(user) {
  return jwt.sign(
    { sub: user.id, name: user.name, email: user.email, role: user.role, shop: user.shop_name || null },
    SECRET, { algorithm: 'HS256', expiresIn: '8h' });
}

function authenticate(req, res, next) {
  const h = req.headers.authorization || '';
  const m = /^Bearer (.+)$/.exec(h);
  if (!m) return next(new HttpError(401, 'Please sign in'));
  try {
    const p = jwt.verify(m[1], SECRET, { algorithms: ['HS256'] });
    req.user = { id: p.sub, name: p.name, email: p.email, role: p.role, shop: p.shop };
    next();
  } catch {
    next(new HttpError(401, 'Your session has expired, please sign in again'));
  }
```

```
}

const requireRole = (...roles) => (req, res, next) =>
  roles.includes(req.user?.role) ? next() : next(new HttpError(403, `This action requires the ${roles.join('
      / ')} role`));

function requireInternal(req, res, next) {
  const given = Buffer.from(String(req.headers['x-internal-key'] || ''));
  const want = Buffer.from(INTERNAL_KEY);
  if (given.length === want.length && crypto.timingSafeEqual(given, want)) return next();
  next(new HttpError(403, 'Internal endpoint'));
}

module.exports = { sign, authenticate, requireRole, requireInternal, INTERNAL_KEY };
```

4.3 shared/db.js

```
/**
 * Each microservice owns its own SQLite database file (database-per-service pattern).
 * Uses Node's built-in node:sqlite (Node 22.13+), so nothing needs compiling.
 */
const { DatabaseSync } = require('node:sqlite');
const fs = require('fs');
const path = require('path');

function open(name) {
  const dir = process.env.DATA_DIR || path.join(__dirname, '..', 'data');
  fs.mkdirSync(dir, { recursive: true });
  const db = new DatabaseSync(path.join(dir, `${name}.db`));
  db.exec('PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON; PRAGMA busy_timeout = 5000;');
  return db;
}

/** Runs fn inside a transaction; rolls back if it throws. */
function tx(db, fn) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = fn();
    db.exec('COMMIT');
    return result;
  } catch (e) {
    try { db.exec('ROLLBACK'); } catch { /* already rolled back */ }
    throw e;
  }
}

module.exports = { open, tx };
```

4.4 gateway/index.js

```
/**
 * API gateway: the single public entry point.
 *  - serves the storefront (static files)
 *  - routes /api/* to the right microservice
 *  - adds a request id, basic rate limiting, health aggregation
 *  - never exposes the services' /internal endpoints
 */
const express = require('express');
const crypto = require('crypto');
const path = require('path');
const { logger, requestLogger, errorHandler } = require('../shared/util');

const log = logger('gateway');
const PORT = +process.env.PORT || 4000;
const USER = process.env.USER_URL || 'http://127.0.0.1:4001';
const CATALOG = process.env.CATALOG_URL || 'http://127.0.0.1:4002';
const ORDER = process.env.ORDER_URL || 'http://127.0.0.1:4003';

// first matching prefix wins (more specific prefixes first)
const ROUTES = [
  ['/auth', 'user-service', USER],
  ['/vendor/products', 'catalog-service', CATALOG],
  ['/products', 'catalog-service', CATALOG],
  ['/categories', 'catalog-service', CATALOG],
  ['/cart', 'order-service', ORDER],
  ['/coupons', 'order-service', ORDER],
  ['/checkout', 'order-service', ORDER],
  ['/orders', 'order-service', ORDER],
  ['/vendor/orders', 'order-service', ORDER],
  ['/vendor/items', 'order-service', ORDER],
  ['/vendor/stats', 'order-service', ORDER],
];
const app = express();
```

```
app.disable('x-powered-by');
app.use((req, res, next) => {
  req.headers['x-request-id'] = crypto.randomBytes(6).toString('hex');
  res.setHeader('X-Request-Id', req.headers['x-request-id']);
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('Referrer-Policy', 'same-origin');
  next();
});
app.use('/api', requestLogger(log));
app.use(express.json({ limit: '50kb' }));

// simple per-IP rate limit: 300 requests / minute
const hits = new Map();
setInterval(() => hits.clear(), 60_000).unref();
app.use('/api', (req, res, next) => {
  const n = (hits.get(req.ip) || 0) + 1;
  hits.set(req.ip, n);
  if (n > 300) return res.status(429).json({ error: 'Too many requests. Slow down a little.' });
  next();
});

app.get('/api/health', async (req, res) => {
  const targets = [['user-service', USER], ['catalog-service', CATALOG], ['order-service', ORDER]];
  const services = await Promise.all(targets.map(async ([name, url]) => {
    try {
      const r = await fetch(url + '/health', { signal: AbortSignal.timeout(1500) });
      return { name, status: r.ok ? 'UP' : 'DOWN' };
    } catch { return { name, status: 'DOWN' }; }
  }));
  res.json({ gateway: 'UP', services, ok: services.every(s => s.status === 'UP') });
});

app.use('/api', async (req, res, next) => {
  const route = ROUTES.find(([p]) => req.path === p || req.path.startsWith(p + '/'));
  if (!route) return next();
  const [, service, base] = route;
  const headers = { 'x-request-id': req.headers['x-request-id'] };
  if (req.headers.authorization) headers.authorization = req.headers.authorization;   // only whitelisted
      headers are forwarded
  if (req.headers['x-forwarded-for']) headers['x-forwarded-for'] = req.headers['x-forwarded-for'];
  else headers['x-forwarded-for'] = req.ip;
  const init = { method: req.method, headers, signal: AbortSignal.timeout(15000) };
  if (!['GET', 'HEAD'].includes(req.method) && req.body && Object.keys(req.body).length) {
    init.body = JSON.stringify(req.body);
    headers['content-type'] = 'application/json';
  }
  try {
    const r = await fetch(base + req.originalUrl.replace(/^\/api/, ''), init);
    res.status(r.status);
    for (const h of ['content-type', 'content-disposition']) if (r.headers.get(h)) res.setHeader(h,
        r.headers.get(h));
    res.send(Buffer.from(await r.arrayBuffer()));
  } catch (e) {
    log('ERROR', `${service} unreachable: ${e.message}`);
    res.status(503).json({ error: `The ${service} is temporarily unavailable. Please try again in a moment.`,
         service });
  }
});

app.use('/api', (req, res) => res.status(404).json({ error: 'Not found' }));
app.use(express.static(path.join(__dirname, 'public'), { extensions: ['html'] }));
app.use(errorHandler(log));

app.listen(PORT, () => log('INFO', `gateway + storefront on http://localhost:${PORT}`));
```

4.5 services/user-service/index.js

```
/**
 * User service: registration, login (JWT), roles (customer / vendor).
 */
const express = require('express');
const crypto = require('crypto');
const { open } = require('../../shared/db');
const auth = require('../../shared/auth');
const { HttpError, bad, logger, requestLogger, cleanStr, start } = require('../../shared/util');

const log = logger('user-service');
const db = open('users');
db.exec(`
CREATE TABLE IF NOT EXISTS users (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  name          TEXT NOT NULL,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
```

```
  salt          TEXT NOT NULL,
  role          TEXT NOT NULL CHECK (role IN ('customer','vendor')),
  shop_name     TEXT,
  created_at    INTEGER NOT NULL
);`);

const q = {
  byEmail: db.prepare('SELECT * FROM users WHERE email = ?'),
  byId: db.prepare('SELECT * FROM users WHERE id = ?'),
  insert: db.prepare('INSERT INTO users (name, email, password_hash, salt, role, shop_name, created_at)
      VALUES (?, ?, ?, ?, ?, ?, ?)'),
  vendors: db.prepare("SELECT id, shop_name FROM users WHERE role = 'vendor' ORDER BY shop_name"),
};

const hash = (pw, salt) => crypto.scryptSync(pw, salt, 64).toString('hex');
const pub = u => ({ id: u.id, name: u.name, email: u.email, role: u.role, shopName: u.shop_name });

// very small in-memory login throttle: 10 failures / 15 min per ip+email
const attempts = new Map();
function throttle(key) {
  const now = Date.now();
  const a = attempts.get(key);
  if (a && a.reset > now && a.count >= 10) throw new HttpError(429, 'Too many failed attempts. Try again in a
       few minutes.');
}
function fail(key) {
  const now = Date.now();
  const a = attempts.get(key);
  if (!a || a.reset < now) attempts.set(key, { count: 1, reset: now + 15 * 60 * 1000 });
  else a.count++;
}

const app = express();
app.use(express.json({ limit: '50kb' }));
app.use(requestLogger(log));

app.get('/health', (req, res) => res.json({ service: 'user-service', status: 'UP' }));

app.post('/auth/register', (req, res) => {
  const b = req.body || {};
  const name = cleanStr(b.name, { min: 2, max: 60, field: 'Name', required: true });
  const email = cleanStr(b.email, { max: 120, field: 'Email', required: true }).toLowerCase();
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) throw bad('Enter a valid email address');
  const password = typeof b.password === 'string' ? b.password : '';
  if (password.length < 8 || password.length > 100) throw bad('Password must be 8 to 100 characters');
  const role = b.role === 'vendor' ? 'vendor' : 'customer';
  const shop = role === 'vendor' ? cleanStr(b.shopName, { min: 2, max: 50, field: 'Shop name', required: true
       }) : null;
  if (q.byEmail.get(email)) throw new HttpError(409, 'An account with this email already exists');
  const salt = crypto.randomBytes(16).toString('hex');
  const info = q.insert.run(name, email, hash(password, salt), salt, role, shop, Date.now());
  const user = q.byId.get(info.lastInsertRowid);
  log('INFO', `registered ${role} ${email}`);
  res.status(201).json({ token: auth.sign(user), user: pub(user) });
});

app.post('/auth/login', (req, res) => {
  const email = String(req.body?.email || '').trim().toLowerCase();
  const password = String(req.body?.password || '');
  const key = `${req.headers['x-forwarded-for'] || req.ip}|${email}`;
  throttle(key);
  const user = q.byEmail.get(email);
  const ok = user && crypto.timingSafeEqual(Buffer.from(hash(password, user.salt), 'hex'),
      Buffer.from(user.password_hash, 'hex'));
  if (!ok) { fail(key); throw new HttpError(401, 'Incorrect email or password'); }
  attempts.delete(key);
  res.json({ token: auth.sign(user), user: pub(user) });
});

app.get('/auth/me', auth.authenticate, (req, res) => {
  const user = q.byId.get(req.user.id);
  if (!user) throw new HttpError(401, 'Account no longer exists');
  res.json({ user: pub(user) });
});

app.get('/auth/vendors', (req, res) => res.json(q.vendors.all().map(v => ({ id: v.id, shopName: v.shop_name
    }))));

start({ name: 'user-service', app, log, port: +process.env.PORT || 4001 });
```

4.6 services/catalog-service/index.js

```
/**
 * Catalog service: products, search / filtering / sorting / pagination,
 * vendor product management and internal stock reservation for the order service.
 * All money is stored as integer paise (1 rupee = 100 paise) to avoid float errors.
 */
const express = require('express');
const { open, tx } = require('../../shared/db');
const auth = require('../../shared/auth');
const { HttpError, bad, logger, requestLogger, cleanStr, cleanInt, clampInt, start } =
    require('../../shared/util');

const log = logger('catalog-service');
const db = open('catalog');
db.exec(`
CREATE TABLE IF NOT EXISTS products (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  vendor_id   INTEGER NOT NULL,
  vendor_name TEXT NOT NULL,
  name        TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  category    TEXT NOT NULL,
  price_paise INTEGER NOT NULL CHECK (price_paise > 0),
  stock       INTEGER NOT NULL CHECK (stock >= 0),
  image_url   TEXT NOT NULL DEFAULT '',
  active      INTEGER NOT NULL DEFAULT 1,
  created_at  INTEGER NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_products_vendor ON products(vendor_id);
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category, active);`);

const map = r => r && ({
  id: r.id, vendorId: r.vendor_id, vendorName: r.vendor_name, name: r.name, description: r.description,
  category: r.category, pricePaise: r.price_paise, stock: r.stock, imageUrl: r.image_url,
  active: !!r.active, createdAt: r.created_at,
});

const SORTS = {
  newest: 'created_at DESC, id DESC', price_asc: 'price_paise ASC, id ASC', price_desc: 'price_paise DESC, id
       ASC',
  name: 'name COLLATE NOCASE ASC', stock: 'stock DESC, id ASC',
};

function where(query, skip = {}) {
  const w = ['active = 1'], p = [];
  const search = String(query.search || '').replace(/[%_\\]/g, ' ').trim().slice(0, 60);
  if (search) { w.push('(name LIKE ? OR description LIKE ? OR category LIKE ? OR vendor_name LIKE ?)'); const
       s = `%${search}%`; p.push(s, s, s, s); }
  if (!skip.category && query.category) { w.push('category = ?'); p.push(String(query.category)); }
  if (!skip.vendor && query.vendor) { w.push('vendor_id = ?'); p.push(clampInt(query.vendor, 0, 0, 1e9)); }
  const min = Number(query.minPrice), max = Number(query.maxPrice);
  if (query.minPrice !== undefined && query.minPrice !== '' && Number.isFinite(min) && min >= 0) {
      w.push('price_paise >= ?'); p.push(Math.round(min * 100)); }
  if (query.maxPrice !== undefined && query.maxPrice !== '' && Number.isFinite(max) && max > 0) {
      w.push('price_paise <= ?'); p.push(Math.round(max * 100)); }
  if (query.inStock === '1' || query.inStock === 'true') w.push('stock > 0');
  return { sql: w.join(' AND '), p };
}

function readProduct(body, partial) {
  const out = {};
  const need = !partial;
  if (need || body.name !== undefined) out.name = cleanStr(body.name, { min: 2, max: 120, field: 'Product
      name', required: true });
  if (body.description !== undefined || need) out.description = cleanStr(body.description, { max: 1000,
      field: 'Description' });
  if (need || body.category !== undefined) out.category = cleanStr(body.category, { min: 2, max: 40, field:
      'Category', required: true });
  if (need || body.pricePaise !== undefined) out.pricePaise = cleanInt(body.pricePaise, { min: 100, max: 1e9,
       field: 'Price (paise)' });
  if (need || body.stock !== undefined) out.stock = cleanInt(body.stock, { min: 0, max: 100000, field:
      'Stock' });
  if (body.imageUrl !== undefined) {
    const u = cleanStr(body.imageUrl, { max: 500, field: 'Image URL' });
    if (u && !/^https?:\/\/[^\s]+$/i.test(u)) throw bad('Image URL must start with http:// or https://');
    out.imageUrl = u;
  }
  return out;
}

const app = express();
app.use(express.json({ limit: '50kb' }));
app.use(requestLogger(log));
app.get('/health', (req, res) => res.json({ service: 'catalog-service', status: 'UP', products:
```

```
    db.prepare('SELECT COUNT(*) n FROM products WHERE active = 1').get().n }));

// ---------- public catalogue ----------
app.get('/products', (req, res) => {
  const page = clampInt(req.query.page, 1, 1, 10000);
  const limit = clampInt(req.query.limit, 12, 1, 48);
  const order = SORTS[req.query.sort] || SORTS.newest;
  const { sql, p } = where(req.query);
  const total = db.prepare(`SELECT COUNT(*) n FROM products WHERE ${sql}`).get(...p).n;
  const items = db.prepare(`SELECT * FROM products WHERE ${sql} ORDER BY ${order} LIMIT ? OFFSET
      ?`).all(...p, limit, (page - 1) * limit).map(map);
  const fc = where(req.query, { category: true });
  const fv = where(req.query, { vendor: true });
  const categories = db.prepare(`SELECT category AS name, COUNT(*) AS count FROM products WHERE ${fc.sql}
      GROUP BY category ORDER BY count DESC, name`).all(...fc.p);
  const vendors = db.prepare(`SELECT vendor_id AS id, vendor_name AS name, COUNT(*) AS count FROM products
      WHERE ${fv.sql} GROUP BY vendor_id ORDER BY name`).all(...fv.p);
  res.json({ items, total, page, pages: Math.max(1, Math.ceil(total / limit)), facets: { categories, vendors
      } });
});

app.get('/products/:id', (req, res) => {
  const r = db.prepare('SELECT * FROM products WHERE id = ? AND active = 1').get(clampInt(req.params.id, 0,
      0, 1e12));
  if (!r) throw new HttpError(404, 'Product not found');
  res.json(map(r));
});

app.get('/categories', (req, res) =>
  res.json(db.prepare('SELECT category AS name, COUNT(*) AS count FROM products WHERE active = 1 GROUP BY
      category ORDER BY count DESC').all()));

// ---------- vendor management ----------
const vendorOnly = [auth.authenticate, auth.requireRole('vendor')];

app.get('/vendor/products', ...vendorOnly, (req, res) =>
  res.json(db.prepare('SELECT * FROM products WHERE vendor_id = ? AND active = 1 ORDER BY id
      DESC').all(req.user.id).map(map)));

app.post('/products', ...vendorOnly, (req, res) => {
  const p = readProduct(req.body || {}, false);
  const info = db.prepare(`INSERT INTO products (vendor_id, vendor_name, name, description, category,
      price_paise, stock, image_url, created_at)
                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`)
    .run(req.user.id, req.user.shop || req.user.name, p.name, p.description, p.category, p.pricePaise,
       p.stock, p.imageUrl || '', Date.now());
  log('INFO', `vendor ${req.user.id} created product ${info.lastInsertRowid}`);
  res.status(201).json(map(db.prepare('SELECT * FROM products WHERE id = ?').get(info.lastInsertRowid)));
});

function ownProduct(req) {
  const r = db.prepare('SELECT * FROM products WHERE id = ? AND active = 1').get(clampInt(req.params.id, 0,
      0, 1e12));
  if (!r) throw new HttpError(404, 'Product not found');
  if (r.vendor_id !== req.user.id) throw new HttpError(403, 'You can only change your own products');
  return r;
}

app.put('/products/:id', ...vendorOnly, (req, res) => {
  const cur = ownProduct(req);
  const p = readProduct(req.body || {}, true);
  const m = {
    name: p.name ?? cur.name, description: p.description ?? cur.description, category: p.category ??
       cur.category,
    price: p.pricePaise ?? cur.price_paise, stock: p.stock ?? cur.stock, image: p.imageUrl ?? cur.image_url,
  };
  db.prepare('UPDATE products SET name=?, description=?, category=?, price_paise=?, stock=?, image_url=?
      WHERE id=?')
    .run(m.name, m.description, m.category, m.price, m.stock, m.image, cur.id);
  res.json(map(db.prepare('SELECT * FROM products WHERE id = ?').get(cur.id)));
});

app.delete('/products/:id', ...vendorOnly, (req, res) => {
  const cur = ownProduct(req);
  db.prepare('UPDATE products SET active = 0 WHERE id = ?').run(cur.id); // soft delete keeps order history
      valid
  res.json({ ok: true });
});

// ---------- internal API (order service only; not exposed by the gateway) ----------
const internal = express.Router();
internal.use(auth.requireInternal);

internal.get('/products', (req, res) => {
  const ids = String(req.query.ids || '').split(',').map(Number).filter(n => Number.isInteger(n) && n >
      0).slice(0, 100);
```

```
  if (!ids.length) return res.json([]);
  res.json(db.prepare(`SELECT * FROM products WHERE id IN (${ids.map(() =>
      '?').join(',')})`).all(...ids).map(map));
});

// Atomically checks stock for every line and decrements it, or changes nothing.
internal.post('/reserve', (req, res) => {
  const items = Array.isArray(req.body?.items) ? req.body.items : [];
  if (!items.length || items.length > 50) throw bad('items required');
  const result = tx(db, () => {
    const lines = [], problems = [];
    for (const it of items) {
      const id = cleanInt(it.productId, { min: 1, field: 'productId' });
      const qty = cleanInt(it.qty, { min: 1, max: 100, field: 'qty' });
      const p = db.prepare('SELECT * FROM products WHERE id = ?').get(id);
      if (!p || !p.active) problems.push({ productId: id, name: p?.name || `#${id}`, reason: 'no longer
          available', available: 0 });
      else if (p.stock < qty) problems.push({ productId: id, name: p.name, reason: 'not enough stock',
          available: p.stock });
      else lines.push({ p, qty });
    }
    if (problems.length) throw new HttpError(409, 'Some items are unavailable', problems);
    for (const { p, qty } of lines) db.prepare('UPDATE products SET stock = stock - ? WHERE id = ?').run(qty,
        p.id);
    return lines.map(({ p, qty }) => ({ productId: p.id, name: p.name, vendorId: p.vendor_id, vendorName:
        p.vendor_name, pricePaise: p.price_paise, qty }));
  });
  log('INFO', `reserved stock for ${result.length} line(s)`);
  res.json({ items: result });
});

internal.post('/release', (req, res) => {
  const items = Array.isArray(req.body?.items) ? req.body.items : [];
  tx(db, () => {
    for (const it of items) {
      db.prepare('UPDATE products SET stock = stock + ? WHERE id = ?')
        .run(cleanInt(it.qty, { min: 1, max: 100, field: 'qty' }), cleanInt(it.productId, { min: 1, field:
            'productId' }));
    }
  });
  log('INFO', `released stock for ${items.length} line(s)`);
  res.json({ ok: true });
});
app.use('/internal', internal);

start({ name: 'catalog-service', app, log, port: +process.env.PORT || 4002 });
```

4.7 services/order-service/pricing.js

```
/**
 * Cart pricing rules (single source of truth for the cart view AND checkout).
 * All amounts are integer paise.
 */
const FREE_SHIPPING_FROM = 99900;  // free shipping when payable amount (before tax) >= Rs. 999
const SHIPPING_FEE = 4900;         // Rs. 49 otherwise
const GST_RATE = 0.18;

const COUPONS = {
  WELCOME10: { type: 'percent', value: 10, min: 0, label: '10% off your order' },
  FLAT100: { type: 'flat', value: 10000, min: 50000, label: 'Rs. 100 off on orders above Rs. 500' },
  FREESHIP: { type: 'shipping', value: 0, min: 0, label: 'Free shipping' },
};

/** lines: [{ pricePaise, qty }] */
function price(lines, couponCode) {
  const subtotal = lines.reduce((s, l) => s + l.pricePaise * l.qty, 0);
  let discount = 0, freeShip = false, coupon = null, couponError = null;

  const code = String(couponCode || '').trim().toUpperCase();
  if (code) {
    const c = COUPONS[code];
    if (!c) couponError = 'Invalid coupon code';
    else if (subtotal < c.min) couponError = `${code} needs a cart value of at least Rs. ${c.min / 100}`;
    else {
      coupon = { code, label: c.label };
      if (c.type === 'percent') discount = Math.round(subtotal * c.value / 100);
      if (c.type === 'flat') discount = Math.min(c.value, subtotal);
      if (c.type === 'shipping') freeShip = true;
    }
  }

  const taxable = subtotal - discount;
  const shipping = subtotal === 0 || freeShip || taxable >= FREE_SHIPPING_FROM ? 0 : SHIPPING_FEE;
  const tax = Math.round(taxable * GST_RATE);
```

```
  return { subtotalPaise: subtotal, discountPaise: discount, shippingPaise: shipping, taxPaise: tax,
           totalPaise: taxable + shipping + tax, coupon, couponError,
           freeShippingFromPaise: FREE_SHIPPING_FROM, gstPercent: GST_RATE * 100 };
}

module.exports = { price, COUPONS, FREE_SHIPPING_FROM, SHIPPING_FEE, GST_RATE };
```

4.8 services/order-service/index.js

```
/**
 * Order service: shopping cart, pricing/coupons, checkout, order tracking,
 * vendor order management + sales statistics, PDF invoices.
 * Talks to the catalog service over HTTP (internal API) to read prices and reserve stock.
 */
const express = require('express');
const crypto = require('crypto');
const { open, tx } = require('../../shared/db');
const auth = require('../../shared/auth');
const { HttpError, bad, logger, requestLogger, cleanStr, cleanInt, clampInt, start } =
    require('../../shared/util');
const { price, COUPONS } = require('./pricing');
const { renderInvoice } = require('./invoice');

const log = logger('order-service');
const CATALOG_URL = process.env.CATALOG_URL || 'http://127.0.0.1:4002';
const FLOW = ['PLACED', 'CONFIRMED', 'PACKED', 'SHIPPED', 'DELIVERED'];
const MAX_LINE_QTY = 20;

const db = open('orders');
db.exec(`
CREATE TABLE IF NOT EXISTS carts (
  user_id    INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  qty        INTEGER NOT NULL CHECK (qty > 0),
  updated_at INTEGER NOT NULL,
  PRIMARY KEY (user_id, product_id)
);
CREATE TABLE IF NOT EXISTS orders (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id        INTEGER NOT NULL,
  customer_name  TEXT NOT NULL,
  customer_email TEXT,
  payment_method TEXT NOT NULL,
  payment_status TEXT NOT NULL,
  transaction_id TEXT,
  coupon_code    TEXT,
  subtotal_paise INTEGER NOT NULL,
  discount_paise INTEGER NOT NULL,
  shipping_paise INTEGER NOT NULL,
  tax_paise      INTEGER NOT NULL,
  total_paise    INTEGER NOT NULL,
  address        TEXT NOT NULL,
  created_at     INTEGER NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_orders_user ON orders(user_id, created_at);
CREATE TABLE IF NOT EXISTS order_items (
  id               INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id         INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id       INTEGER NOT NULL,
  vendor_id        INTEGER NOT NULL,
  vendor_name      TEXT NOT NULL,
  name             TEXT NOT NULL,
  unit_price_paise INTEGER NOT NULL,
  qty              INTEGER NOT NULL,
  status           TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_items_vendor ON order_items(vendor_id);
CREATE INDEX IF NOT EXISTS idx_items_order ON order_items(order_id);
CREATE TABLE IF NOT EXISTS order_events (
  id       INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  item_id  INTEGER,
  status   TEXT NOT NULL,
  note     TEXT,
  actor    TEXT,
  ts       INTEGER NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_events_order ON order_events(order_id);`);

// ---------- catalog client ----------
async function catalog(path, { method = 'GET', body } = {}) {
  let r;
  try {
    r = await fetch(CATALOG_URL + '/internal' + path, {
```

```
      method, signal: AbortSignal.timeout(4000),
      headers: { 'content-type': 'application/json', 'x-internal-key': auth.INTERNAL_KEY },
      body: body ? JSON.stringify(body) : undefined,
    });
  } catch {
    throw new HttpError(503, 'The catalog service is unavailable right now. Please try again shortly.');
  }
  const data = await r.json().catch(() => ({}));
  if (!r.ok) throw new HttpError(r.status === 409 ? 409 : 502, data.error || 'Catalog service error',
      data.details);
  return data;
}

// ---------- cart ----------
const customerOnly = [auth.authenticate, auth.requireRole('customer')];
const vendorOnly = [auth.authenticate, auth.requireRole('vendor')];

async function buildCart(userId, couponCode) {
  const rows = db.prepare('SELECT product_id, qty FROM carts WHERE user_id = ? ORDER BY updated_at
      DESC').all(userId);
  const products = rows.length ? await catalog(`/products?ids=${rows.map(r => r.product_id).join(',')}`) :
      [];
  const byId = new Map(products.map(p => [p.id, p]));
  const items = [];
  for (const r of rows) {
    const p = byId.get(r.product_id);
    if (!p || !p.active) {
      items.push({ productId: r.product_id, name: p?.name || `Product #${r.product_id}`, vendorName:
          p?.vendorName || '', pricePaise: 0, qty: r.qty, stock: 0, imageUrl: '', category: '',
          lineTotalPaise: 0, issue: 'No longer available' });
      continue;
    }
    items.push({
      productId: p.id, name: p.name, vendorName: p.vendorName, category: p.category, imageUrl: p.imageUrl,
      pricePaise: p.pricePaise, qty: r.qty, stock: p.stock, lineTotalPaise: p.pricePaise * r.qty,
      issue: p.stock < r.qty ? (p.stock === 0 ? 'Out of stock' : `Only ${p.stock} left`) : null,
    });
  }
  const priced = price(items.filter(i => !i.issue || i.pricePaise > 0), couponCode);
  return { items, count: items.reduce((s, i) => s + i.qty, 0), hasIssues: items.some(i => i.issue), ...priced
       };
}

const app = express();
app.use(express.json({ limit: '50kb' }));
app.use(requestLogger(log));

app.get('/health', (req, res) => res.json({ service: 'order-service', status: 'UP', orders:
    db.prepare('SELECT COUNT(*) n FROM orders').get().n }));

app.get('/cart', ...customerOnly, async (req, res) => res.json(await buildCart(req.user.id,
    req.query.coupon)));

async function checkedProduct(productId) {
  const [p] = await catalog(`/products?ids=${productId}`);
  if (!p || !p.active) throw new HttpError(404, 'This product is no longer available');
  return p;
}

app.post('/cart/items', ...customerOnly, async (req, res) => {
  const productId = cleanInt(req.body?.productId, { min: 1, field: 'productId' });
  const add = cleanInt(req.body?.qty ?? 1, { min: 1, max: MAX_LINE_QTY, field: 'Quantity' });
  const p = await checkedProduct(productId);
  const cur = db.prepare('SELECT qty FROM carts WHERE user_id = ? AND product_id = ?').get(req.user.id,
      productId)?.qty || 0;
  const qty = cur + add;
  if (p.stock <= 0) throw new HttpError(409, `${p.name} is out of stock`);
  if (qty > p.stock) throw new HttpError(409, `Only ${p.stock} of ${p.name} available (you already have
      ${cur} in your cart)`);
  if (qty > MAX_LINE_QTY) throw new HttpError(409, `You can buy at most ${MAX_LINE_QTY} of one product per
      order`);
  db.prepare(`INSERT INTO carts (user_id, product_id, qty, updated_at) VALUES (?, ?, ?, ?)
              ON CONFLICT(user_id, product_id) DO UPDATE SET qty = excluded.qty, updated_at =
                  excluded.updated_at`)
    .run(req.user.id, productId, qty, Date.now());
  res.status(201).json(await buildCart(req.user.id, req.query.coupon));
});

app.put('/cart/items/:productId', ...customerOnly, async (req, res) => {
  const productId = cleanInt(req.params.productId, { min: 1, field: 'productId' });
  const qty = cleanInt(req.body?.qty, { min: 0, max: MAX_LINE_QTY, field: 'Quantity' });
  if (qty === 0) db.prepare('DELETE FROM carts WHERE user_id = ? AND product_id = ?').run(req.user.id,
      productId);
  else {
    const p = await checkedProduct(productId);
    if (qty > p.stock) throw new HttpError(409, `Only ${p.stock} of ${p.name} available`);
```

```
    db.prepare(`INSERT INTO carts (user_id, product_id, qty, updated_at) VALUES (?, ?, ?, ?)
                ON CONFLICT(user_id, product_id) DO UPDATE SET qty = excluded.qty, updated_at =
                    excluded.updated_at`)
      .run(req.user.id, productId, qty, Date.now());
  }
  res.json(await buildCart(req.user.id, req.query.coupon));
});

app.delete('/cart/items/:productId', ...customerOnly, async (req, res) => {
  db.prepare('DELETE FROM carts WHERE user_id = ? AND product_id = ?').run(req.user.id,
      clampInt(req.params.productId, 0, 0, 1e12));
  res.json(await buildCart(req.user.id, req.query.coupon));
});

app.delete('/cart', ...customerOnly, async (req, res) => {
  db.prepare('DELETE FROM carts WHERE user_id = ?').run(req.user.id);
  res.json(await buildCart(req.user.id));
});

app.get('/coupons', (req, res) => res.json(Object.entries(COUPONS).map(([code, c]) => ({ code, label: c.label
     })));

// ---------- orders ----------
const orderStatus = items => {
  const active = items.filter(i => i.status !== 'CANCELLED');
  if (!active.length) return 'CANCELLED';
  return FLOW[Math.min(...active.map(i => FLOW.indexOf(i.status)))];
};

function loadOrder(id) {
  const o = db.prepare('SELECT * FROM orders WHERE id = ?').get(id);
  if (!o) return null;
  const items = db.prepare('SELECT * FROM order_items WHERE order_id = ? ORDER BY id').all(id).map(i => ({
    id: i.id, productId: i.product_id, vendorId: i.vendor_id, vendorName: i.vendor_name, name: i.name,
    unitPricePaise: i.unit_price_paise, qty: i.qty, lineTotalPaise: i.unit_price_paise * i.qty, status:
        i.status,
  }));
  const events = db.prepare('SELECT id, item_id, status, note, actor, ts FROM order_events WHERE order_id = ?
       ORDER BY ts, id').all(id)
    .map(e => ({ id: e.id, itemId: e.item_id, status: e.status, note: e.note, actor: e.actor, ts: e.ts }));
  return {
    id: o.id, invoiceNo: `INV-${new Date(o.created_at).getFullYear()}-${String(o.id).padStart(5, '0')}`,
    userId: o.user_id, customerName: o.customer_name, status: orderStatus(items),
    paymentMethod: o.payment_method, paymentStatus: o.payment_status, transactionId: o.transaction_id,
        couponCode: o.coupon_code,
    subtotalPaise: o.subtotal_paise, discountPaise: o.discount_paise, shippingPaise: o.shipping_paise,
        taxPaise: o.tax_paise, totalPaise: o.total_paise,
    address: JSON.parse(o.address), createdAt: o.created_at, items, events,
  };
}

function event(orderId, itemId, status, note, actor) {
  db.prepare('INSERT INTO order_events (order_id, item_id, status, note, actor, ts) VALUES (?, ?, ?, ?, ?,
      ?)').run(orderId, itemId, status, note, actor, Date.now());
}

function syncPayment(orderId) {
  const o = db.prepare('SELECT payment_method, payment_status FROM orders WHERE id = ?').get(orderId);
  const items = db.prepare('SELECT status FROM order_items WHERE order_id = ?').all(orderId);
  const active = items.filter(i => i.status !== 'CANCELLED');
  let next = o.payment_status;
  if (!active.length) next = o.payment_status === 'PAID' ? 'REFUNDED' : 'CANCELLED';
  else if (o.payment_method === 'COD' && active.every(i => i.status === 'DELIVERED')) next = 'PAID';
  if (next !== o.payment_status) db.prepare('UPDATE orders SET payment_status = ? WHERE id = ?').run(next,
      orderId);
}

/** Cancels the given order items (only while PLACED/CONFIRMED) and returns stock to the catalog. */
async function cancelItems(orderId, predicate, actor, note) {
  const items = db.prepare("SELECT * FROM order_items WHERE order_id = ? AND status IN
      ('PLACED','CONFIRMED')").all(orderId).filter(predicate);
  if (!items.length) return [];
  await catalog('/release', { method: 'POST', body: { items: items.map(i => ({ productId: i.product_id, qty:
      i.qty })) } });
  tx(db, () => {
    for (const i of items) {
      db.prepare("UPDATE order_items SET status = 'CANCELLED' WHERE id = ?").run(i.id);
      event(orderId, i.id, 'CANCELLED', note, actor);
    }
    syncPayment(orderId);
  });
  return items;
}

function readAddress(a) {
  if (!a || typeof a !== 'object') throw bad('Delivery address is required');
```

```
  const out = {
    name: cleanStr(a.name, { min: 2, max: 60, field: 'Full name', required: true }),
    phone: cleanStr(a.phone, { min: 7, max: 15, field: 'Phone', required: true }),
    line1: cleanStr(a.line1, { min: 5, max: 150, field: 'Address', required: true }),
    city: cleanStr(a.city, { min: 2, max: 50, field: 'City', required: true }),
    state: cleanStr(a.state, { min: 2, max: 50, field: 'State', required: true }),
    pin: cleanStr(a.pin, { min: 6, max: 6, field: 'PIN code', required: true }),
  };
  if (!/^[0-9+\- ]{7,15}$/.test(out.phone)) throw bad('Enter a valid phone number');
  if (!/^[0-9]{6}$/.test(out.pin)) throw bad('PIN code must be 6 digits');
  return out;
}

app.post('/checkout', ...customerOnly, async (req, res) => {
  const b = req.body || {};
  const address = readAddress(b.address);
  const method = ['COD', 'CARD_TEST', 'UPI_TEST'].includes(b.paymentMethod) ? b.paymentMethod : null;
  if (!method) throw bad('Choose a payment method');
  const rows = db.prepare('SELECT product_id, qty FROM carts WHERE user_id = ?').all(req.user.id);
  if (!rows.length) throw bad('Your cart is empty');

  // 1) reserve stock atomically in the catalog service (all lines or nothing)
  const reserved = (await catalog('/reserve', { method: 'POST', body: { items: rows.map(r => ({ productId:
      r.product_id, qty: r.qty })) } })).items;
  const release = () => catalog('/release', { method: 'POST', body: { items: reserved.map(i => ({ productId:
      i.productId, qty: i.qty })) } }).catch(e => log('ERROR', 'stock release failed: ' + e.message));

  try {
    // 2) price from the authoritative snapshot returned by the catalog (never trust client prices)
    const priced = price(reserved, b.coupon);
    if (b.coupon && priced.couponError) throw bad(priced.couponError);
    const paid = method !== 'COD';
    const txn = paid ? 'TXN' + crypto.randomBytes(5).toString('hex').toUpperCase() : null;

    // 3) persist the order, its items and first tracking event; empty the cart
    const orderId = tx(db, () => {
      const info = db.prepare(`INSERT INTO orders (user_id, customer_name, customer_email, payment_method,
          payment_status, transaction_id, coupon_code,
          subtotal_paise, discount_paise, shipping_paise, tax_paise, total_paise, address, created_at) VALUES
               (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`)
        .run(req.user.id, req.user.name, req.user.email || '', method, paid ? 'PAID' : 'PENDING', txn,
            priced.coupon?.code || null,
          priced.subtotalPaise, priced.discountPaise, priced.shippingPaise, priced.taxPaise,
              priced.totalPaise, JSON.stringify(address), Date.now());
      const oid = Number(info.lastInsertRowid);
      for (const i of reserved) {
        const it = db.prepare('INSERT INTO order_items (order_id, product_id, vendor_id, vendor_name, name,
            unit_price_paise, qty, status) VALUES (?, ?, ?, ?, ?, ?, ?, ?)')
          .run(oid, i.productId, i.vendorId, i.vendorName, i.name, i.pricePaise, i.qty, 'PLACED');
        event(oid, Number(it.lastInsertRowid), 'PLACED', `${i.qty} x ${i.name}`, 'customer');
      }
      db.prepare('DELETE FROM carts WHERE user_id = ?').run(req.user.id);
      return oid;
    });
    log('INFO', `order #${orderId} placed by user ${req.user.id} total=${priced.totalPaise}`);
    res.status(201).json(loadOrder(orderId));
  } catch (e) {
    await release();
    throw e;
  }
});

const summary = o => ({
  id: o.id, createdAt: o.createdAt, status: o.status, totalPaise: o.totalPaise, paymentMethod:
      o.paymentMethod, paymentStatus: o.paymentStatus,
  items: o.items.map(i => ({ name: i.name, qty: i.qty, status: i.status, vendorName: i.vendorName })),
});

app.get('/orders', ...customerOnly, (req, res) => {
  const ids = db.prepare('SELECT id FROM orders WHERE user_id = ? ORDER BY created_at DESC, id DESC LIMIT
      100').all(req.user.id);
  res.json(ids.map(r => summary(loadOrder(r.id))));
});

function ownOrder(req) {
  const o = loadOrder(clampInt(req.params.id, 0, 0, 1e12));
  if (!o || o.userId !== req.user.id) throw new HttpError(404, 'Order not found');
  return o;
}

app.get('/orders/:id', ...customerOnly, (req, res) => res.json(ownOrder(req)));

app.post('/orders/:id/cancel', ...customerOnly, async (req, res) => {
  const o = ownOrder(req);
  const done = await cancelItems(o.id, () => true, 'customer', 'Cancelled by customer');
  if (!done.length) throw new HttpError(409, 'This order can no longer be cancelled (it has already been
```

```
      packed or shipped)');
  res.json(loadOrder(o.id));
});

app.get('/orders/:id/invoice', ...customerOnly, (req, res) => {
  const o = ownOrder(req);
  res.setHeader('Content-Type', 'application/pdf');
  res.setHeader('Content-Disposition', `attachment; filename="${o.invoiceNo}.pdf"`);
  renderInvoice(o, res);
});

// ---------- vendor ----------
app.get('/vendor/orders', ...vendorOnly, (req, res) => {
  const rows = db.prepare(`SELECT oi.*, o.created_at, o.customer_name, o.address, o.payment_method,
      o.payment_status
                           FROM order_items oi JOIN orders o ON o.id = oi.order_id
                           WHERE oi.vendor_id = ? ORDER BY o.created_at DESC, oi.id DESC LIMIT
                               200`).all(req.user.id);
  res.json(rows.map(r => ({
    itemId: r.id, orderId: r.order_id, productId: r.product_id, name: r.name, qty: r.qty, unitPricePaise:
        r.unit_price_paise,
    lineTotalPaise: r.unit_price_paise * r.qty, status: r.status, createdAt: r.created_at, customerName:
        r.customer_name,
    address: JSON.parse(r.address), paymentMethod: r.payment_method, paymentStatus: r.payment_status,
    next: FLOW[FLOW.indexOf(r.status) + 1] || null, canCancel: ['PLACED', 'CONFIRMED'].includes(r.status),
  })));
});

app.put('/vendor/items/:id/status', ...vendorOnly, async (req, res) => {
  const item = db.prepare('SELECT * FROM order_items WHERE id = ?').get(clampInt(req.params.id, 0, 0, 1e12));
  if (!item) throw new HttpError(404, 'Order item not found');
  if (item.vendor_id !== req.user.id) throw new HttpError(403, 'This item belongs to another vendor');
  const target = String(req.body?.status || '').toUpperCase();

  if (target === 'CANCELLED') {
    const done = await cancelItems(item.order_id, i => i.id === item.id, 'vendor', `Cancelled by
        ${req.user.shop || 'vendor'}`);
    if (!done.length) throw new HttpError(409, 'This item can no longer be cancelled');
  } else {
    const from = FLOW.indexOf(item.status);
    if (item.status === 'CANCELLED' || from === -1) throw new HttpError(409, 'This item is cancelled');
    if (FLOW.indexOf(target) !== from + 1) throw new HttpError(409, `Next valid status is ${FLOW[from + 1] ||
         'none (already delivered)'}`);
    tx(db, () => {
      db.prepare('UPDATE order_items SET status = ? WHERE id = ?').run(target, item.id);
      event(item.order_id, item.id, target, `${item.name} marked ${target.toLowerCase()}`, req.user.shop ||
          'vendor');
      syncPayment(item.order_id);
    });
  }
  const upd = db.prepare('SELECT status FROM order_items WHERE id = ?').get(item.id);
  res.json({ itemId: item.id, status: upd.status });
});

app.get('/vendor/stats', ...vendorOnly, (req, res) => {
  const v = req.user.id;
  const totals = db.prepare("SELECT COUNT(DISTINCT order_id) AS orders, COALESCE(SUM(qty),0) AS units,
      COALESCE(SUM(qty*unit_price_paise),0) AS revenue FROM order_items WHERE vendor_id = ? AND status !=
      'CANCELLED'").get(v);
  const statusCounts = Object.fromEntries(db.prepare('SELECT status, COUNT(*) AS n FROM order_items WHERE
      vendor_id = ? GROUP BY status').all(v).map(r => [r.status, r.n]));
  const top = db.prepare("SELECT name, SUM(qty) AS units, SUM(qty*unit_price_paise) AS revenue FROM
      order_items WHERE vendor_id = ? AND status != 'CANCELLED' GROUP BY product_id ORDER BY revenue DESC
      LIMIT 5").all(v);
  const rows = db.prepare(`SELECT strftime('%Y-%m-%d', o.created_at/1000, 'unixepoch', 'localtime') AS day,
      SUM(oi.qty*oi.unit_price_paise) AS revenue, COUNT(DISTINCT o.id) AS orders
                           FROM order_items oi JOIN orders o ON o.id = oi.order_id
                           WHERE oi.vendor_id = ? AND oi.status != 'CANCELLED' AND o.created_at >= ? GROUP BY
                                day`).all(v, Date.now() - 8 * 864e5);
  const byDay = new Map(rows.map(r => [r.day, r]));
  const pad = n => String(n).padStart(2, '0');
  const daily = [];
  for (let i = 6; i >= 0; i--) {
    const d = new Date(Date.now() - i * 864e5);
    const key = `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
    daily.push({ day: key, revenuePaise: byDay.get(key)?.revenue || 0, orders: byDay.get(key)?.orders || 0
        });
  }
  res.json({ ordersCount: totals.orders, unitsSold: totals.units, revenuePaise: totals.revenue, statusCounts,
       topProducts: top.map(t => ({ name: t.name, units: t.units, revenuePaise: t.revenue })), daily });
});

start({ name: 'order-service', app, log, port: +process.env.PORT || 4003 });
```

5. Output Screenshots

The screenshots below were taken from the running application at http://localhost:4000.

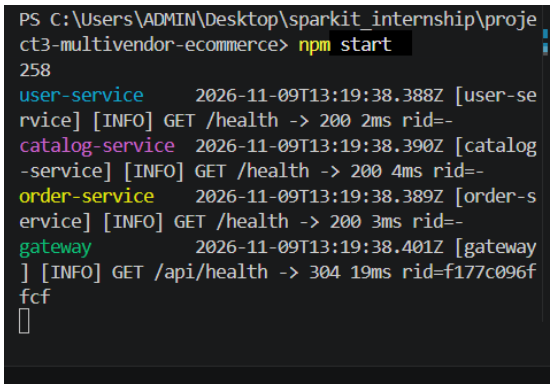


Figure 3: Terminal after npm start. The user, catalog and order services and the gateway are running as separate processes and answering health checks.

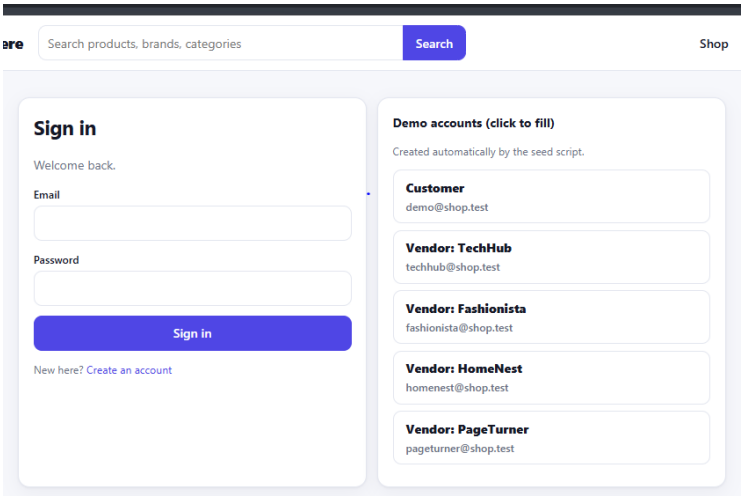


Figure 4: Sign-in page. The demo accounts (one customer, four vendors) are created by the seed script and can be filled in with one click.

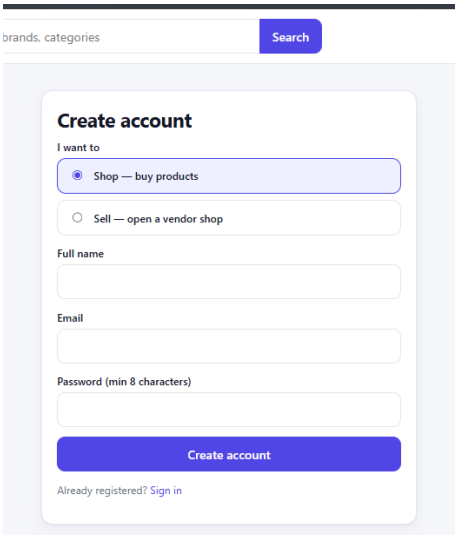


Figure 5: Registration page. A new user chooses to shop or to open a vendor shop; a shop name is asked only for vendors.

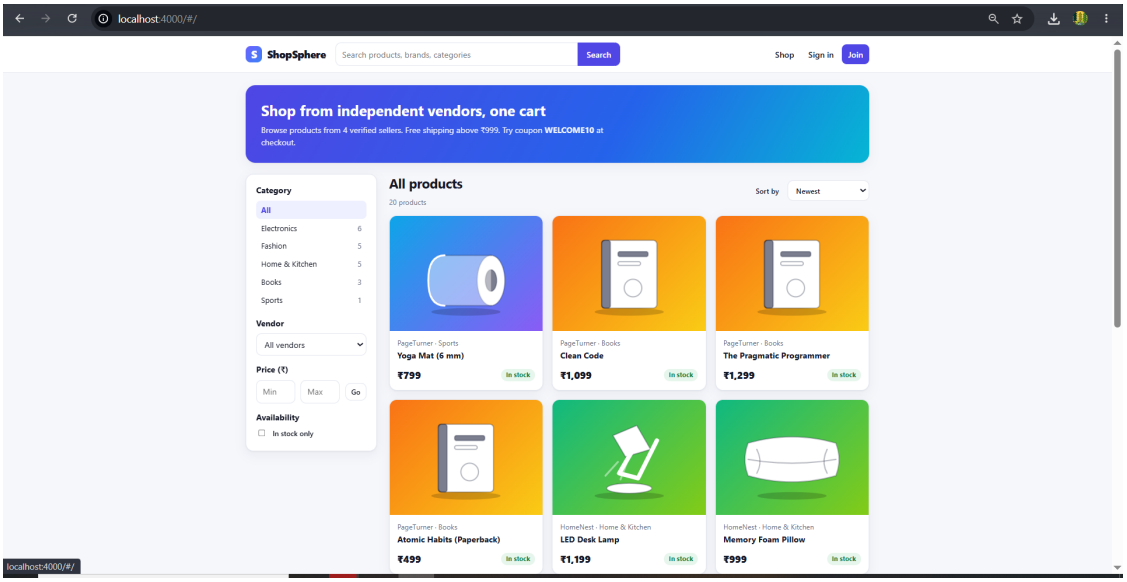


Figure 6: Storefront home page with product listing, category counts, vendor and price filters and the sort control.

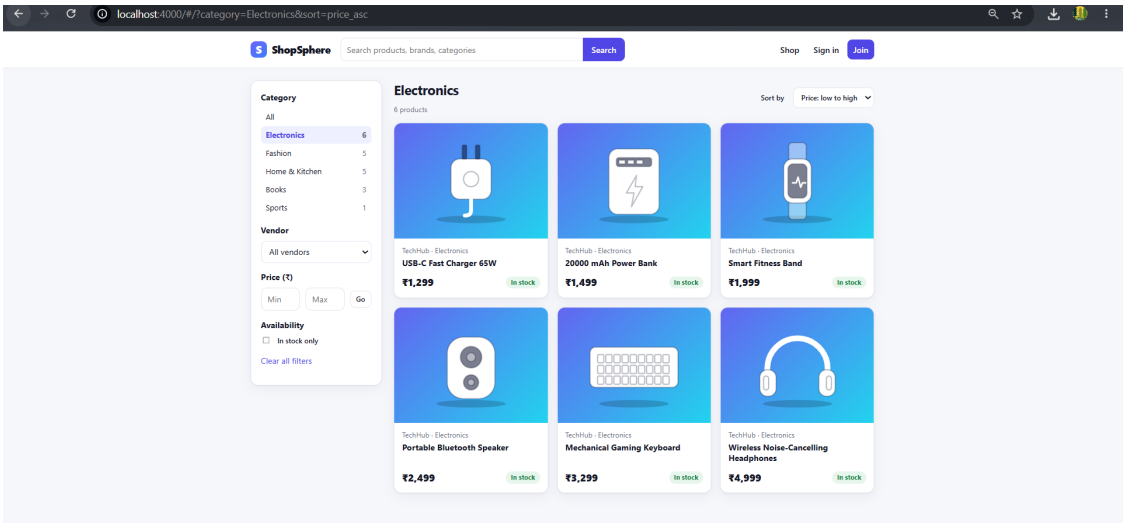


Figure 7: Filtering and sorting: Electronics category sorted by price, low to high (the address bar shows category=Electronics&sort=price_asc).

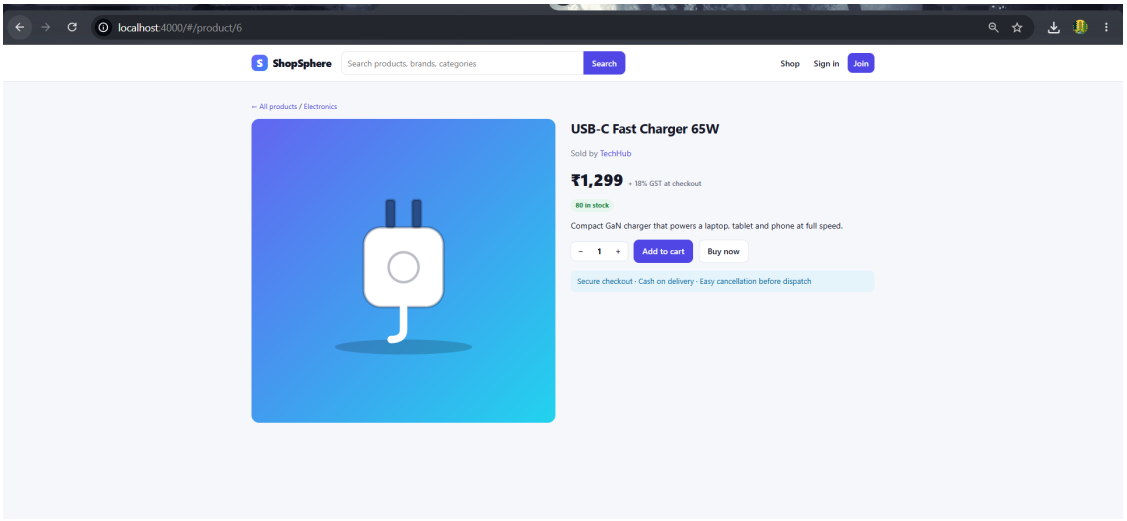


Figure 8: Product detail page with vendor, price, stock status, quantity selector, Add to cart and Buy now.

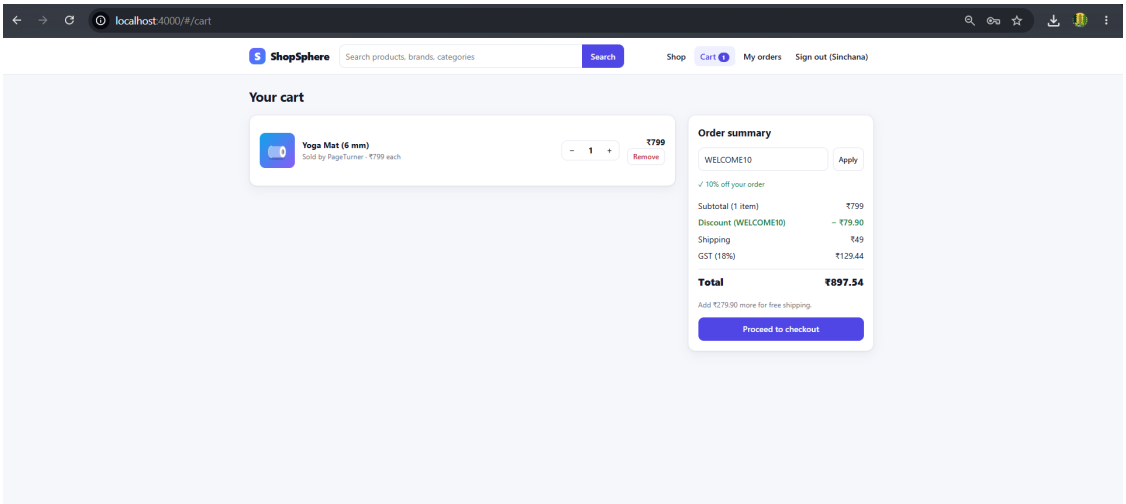


Figure 9: Cart with coupon WELCOME10: subtotal Rs. 799, discount Rs. 79.90, shipping Rs. 49, GST Rs. 129.44, total Rs. 897.54.

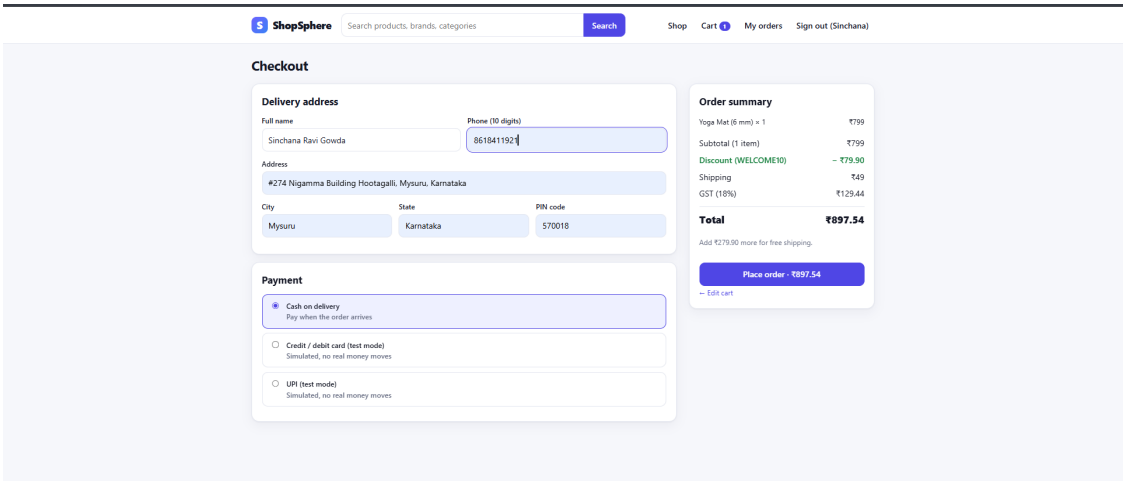


Figure 10: Checkout page with delivery address and payment options (cash on delivery, test card, test UPI).

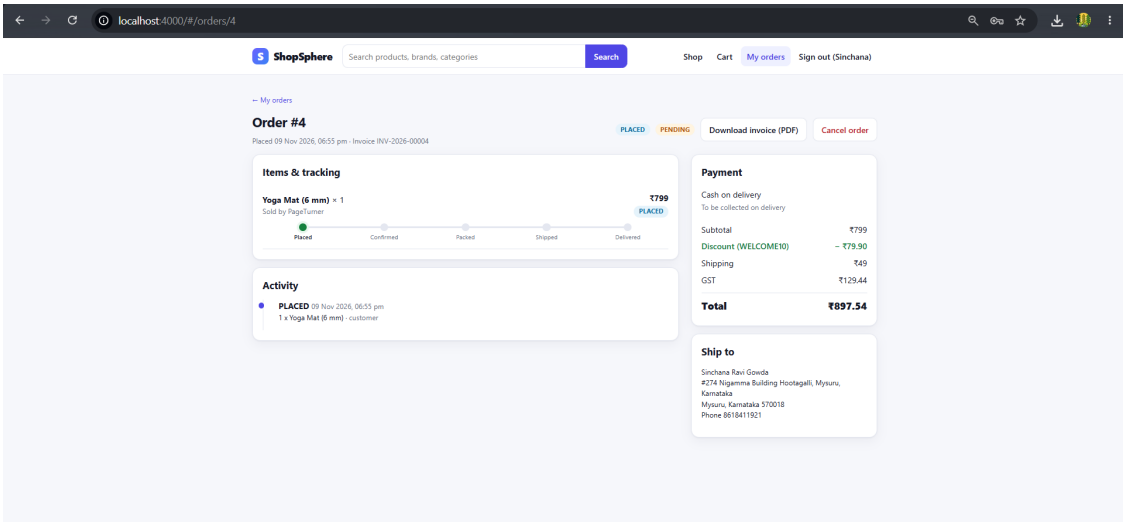
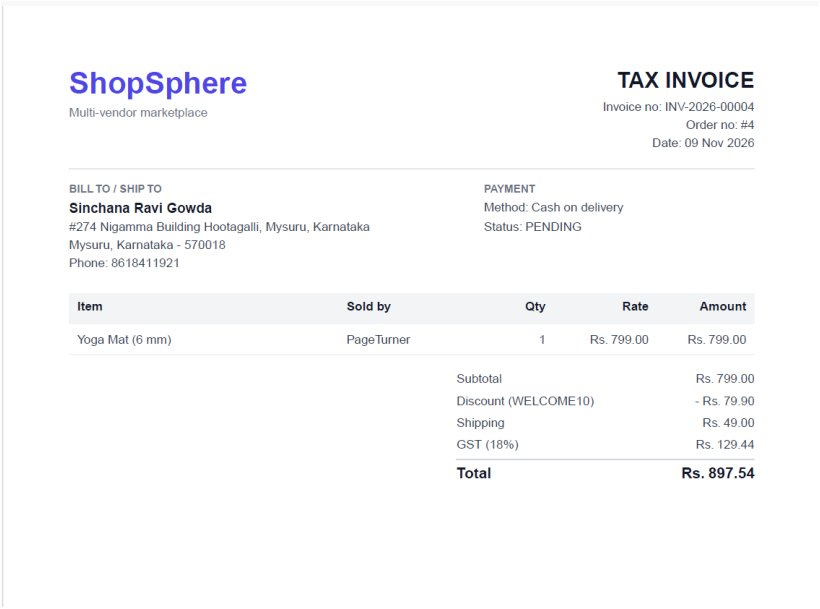


Figure 11: Order page after checkout: item tracking stepper, activity log, payment summary and delivery address, with Download invoice and Cancel order.



ShopSphere
Multi-vendor marketplace

TAX INVOICE
Invoice no: INV-2026-00004
Order no: #4
Date: 09 Nov 2026

BILL TO / SHIP TO
Sinchana Ravi Gowda
#274 Nigamma Building Hootagalli, Mysuru, Karnataka
Mysuru, Karnataka - 570018
Phone: 8618411921

PAYMENT
Method: Cash on delivery
Status: PENDING

Item	Sold by	Qty	Rate	Amount
Yoga Mat (6 mm)	PageTurner	1	Rs. 799.00	Rs. 799.00

Subtotal	Rs. 799.00
Discount (WELCOME10)	- Rs. 79.90
Shipping	Rs. 49.00
GST (18%)	Rs. 129.44
Total	**Rs. 897.54**

Figure 12: PDF tax invoice generated for the order (INV-2026-00004).

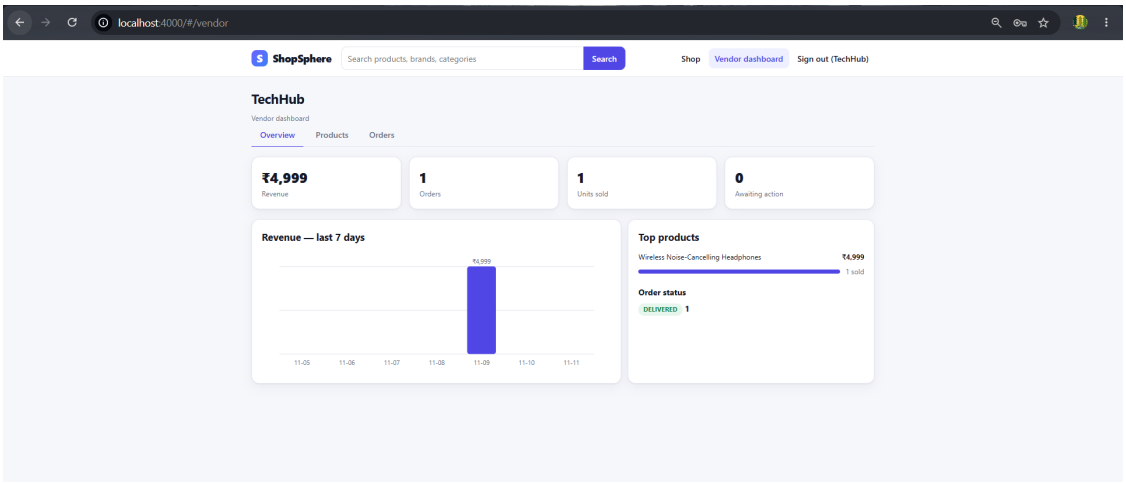


Figure 13: Vendor dashboard of TechHub: revenue, orders, units sold, 7-day revenue chart, top products and order status.

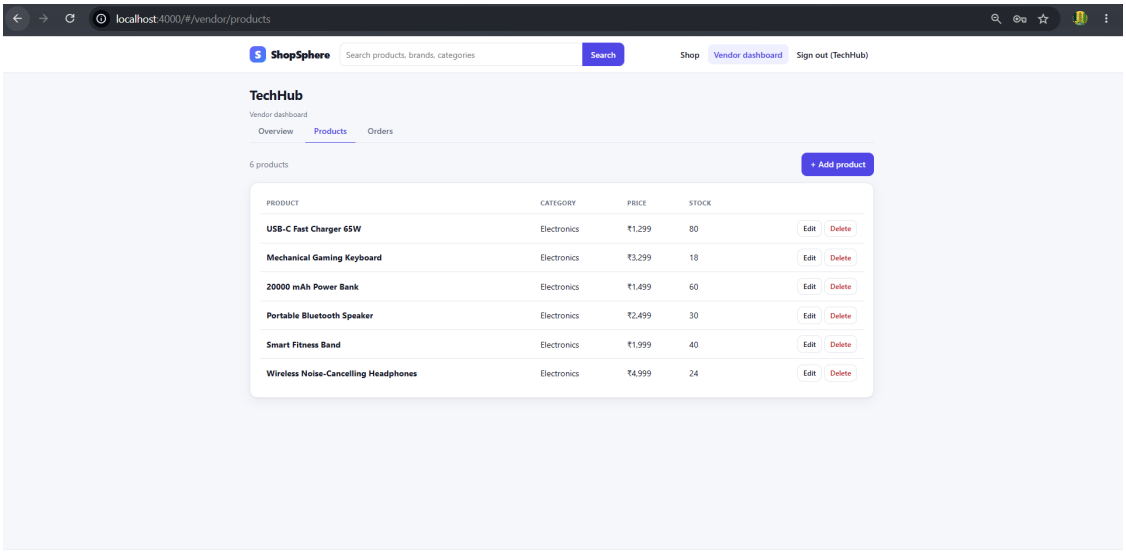


Figure 14: TechHub product management: the vendor's own products with price and stock, and Edit and Delete actions.

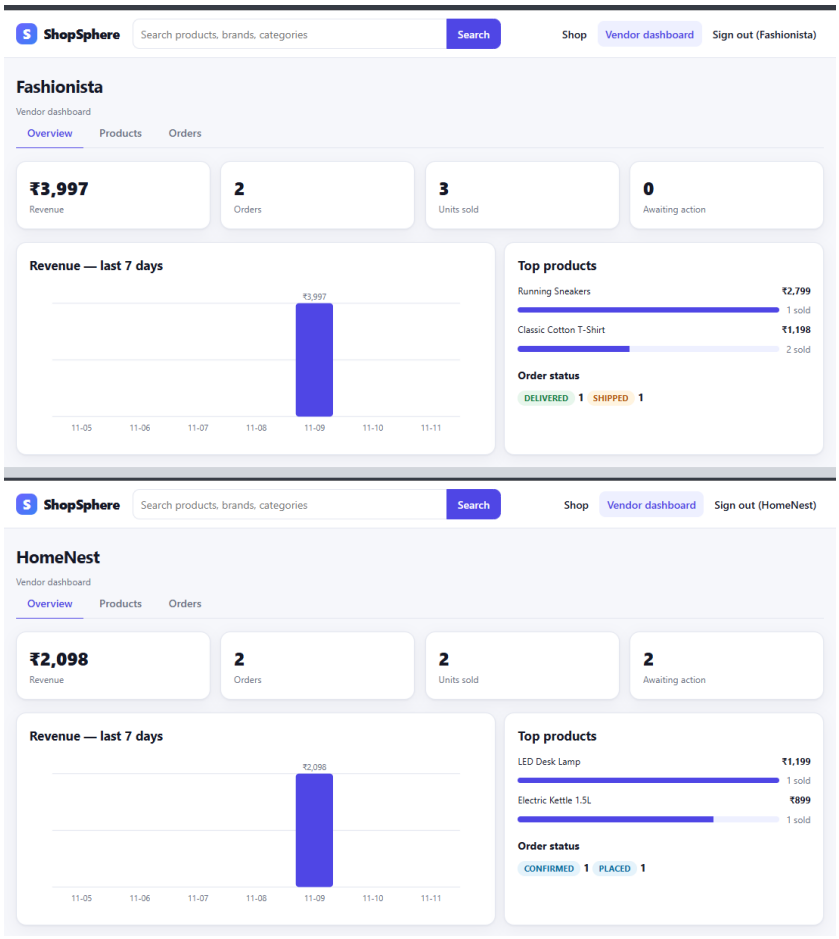


Figure 15: Dashboards of two more vendors, Fashionista (top) and HomeNest (bottom). Each vendor sees only the sales of their own products.

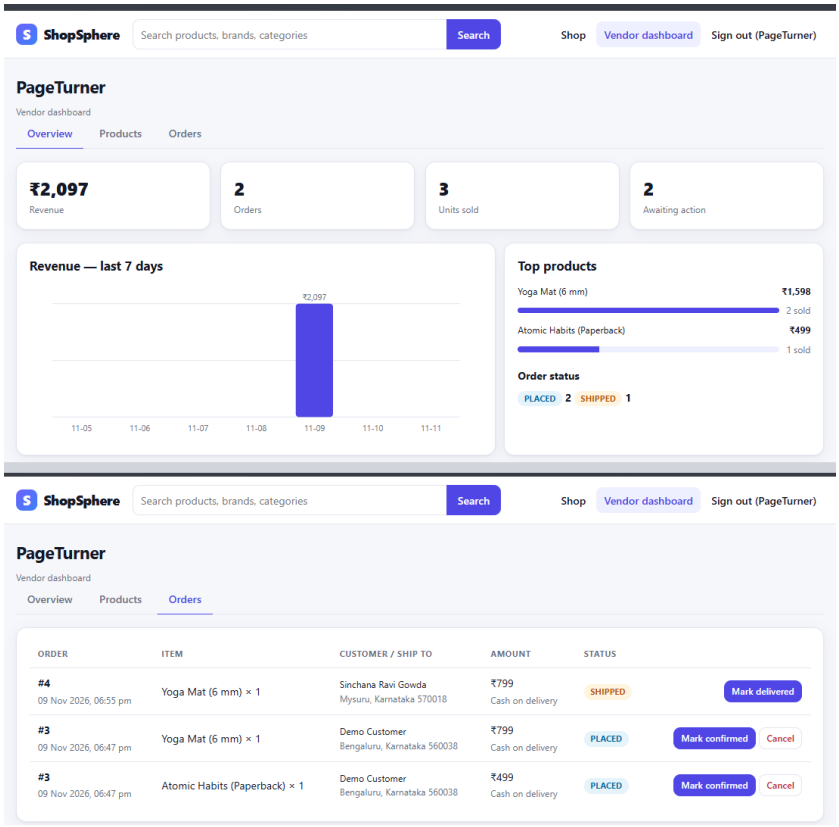


Figure 16: PageTurner dashboard (top) and order list (bottom). The first row is order #4, the real order I placed as a customer, now marked SHIPPED by the vendor.

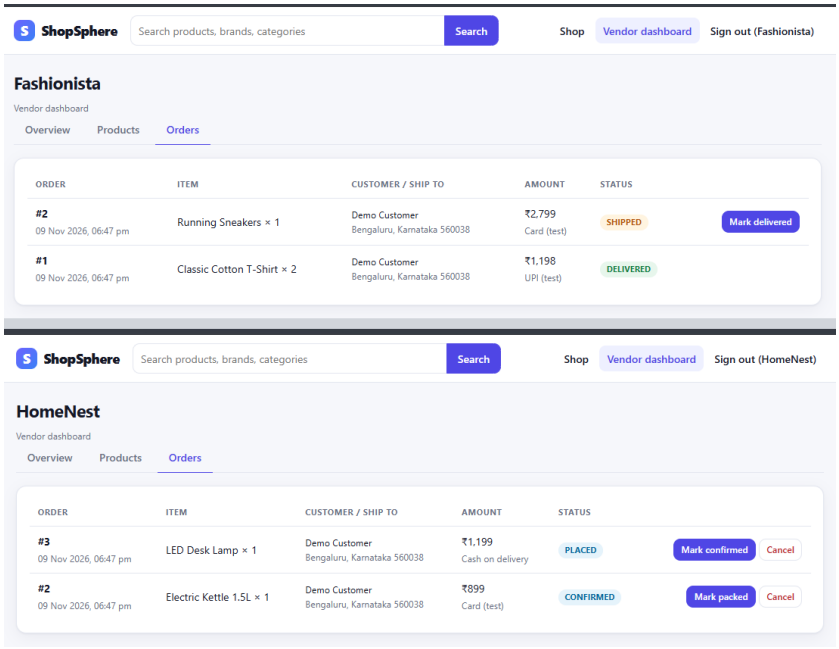


Figure 17: Order fulfilment for Fashionista (top) and HomeNest (bottom): Mark delivered, Mark confirmed, Mark packed and Cancel buttons depend on each item's current status.

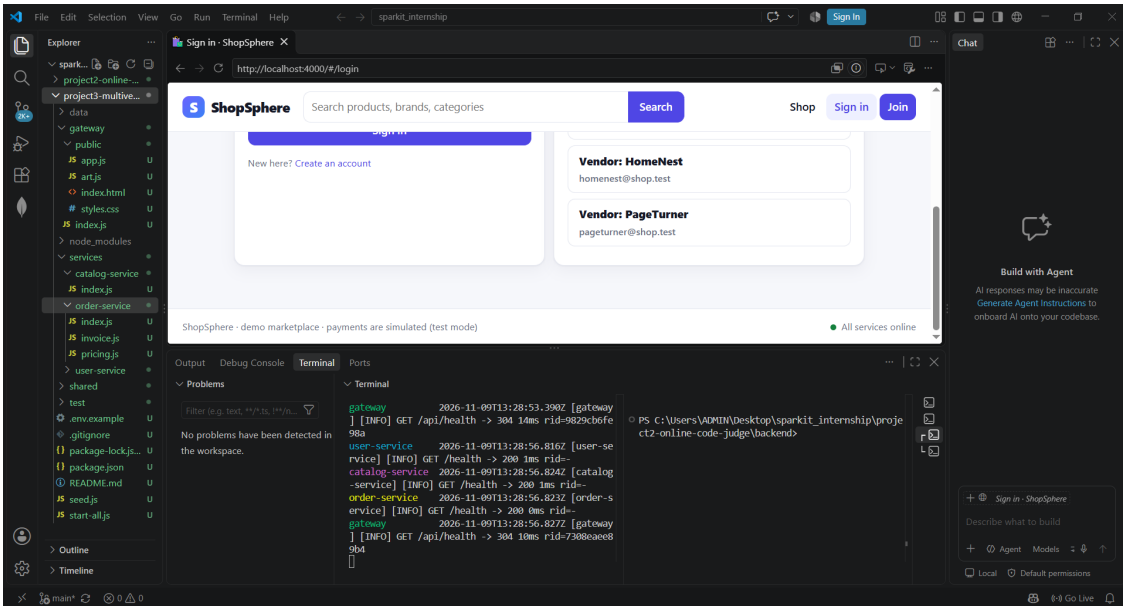


Figure 18: Project structure in VS Code (gateway, services, shared, test, seed.js, start-all.js), the storefront in the built-in browser and the service logs in the terminal.

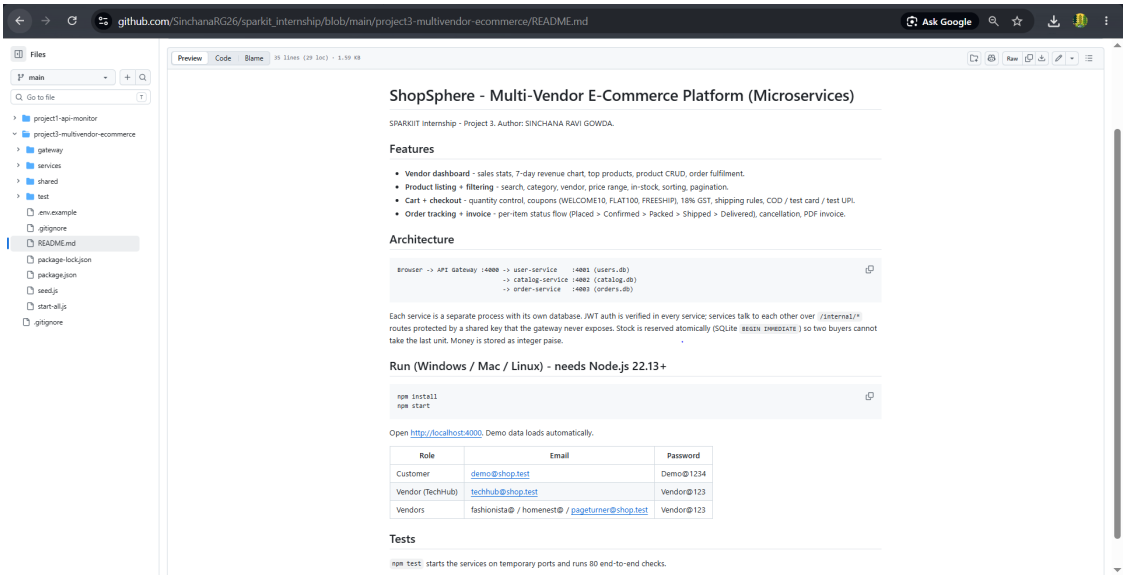


Figure 19: Source code on GitHub (sparkit_internship / project3-multivendor-ecommerce) with the project README. node_modules, the data folder and .env are not uploaded.

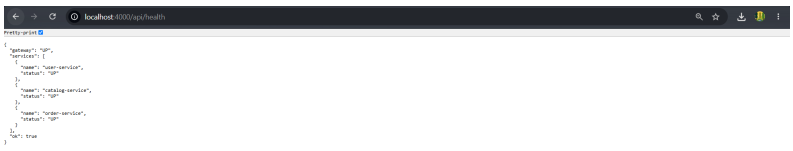


Figure 20: Gateway health endpoint (/api/health) reporting all three services UP.

6. Results and Analysis

6.1 Automated test results

The command `npm test` starts all services on temporary ports with empty databases and runs 80 checks through the gateway. All 80 passed, 0 failed.

Area	Checks	Examples verified
Accounts and security	17	duplicate e-mail 409, weak password 400, wrong password 401, tampered token 401, customer cannot create products 403, other vendor cannot edit product 403
Listing and filtering	10	category, vendor, search, price range, sort ascending and descending, pagination, facets, SQL-injection style search harmless
Cart and pricing	14	stock limit 409, quantity update, subtotal, 10% discount, GST on discounted amount, shipping rule, FREESHIP, invalid coupon
Checkout	11	invalid address 400, failed checkout leaves stock untouched, two-vendor order, server-side prices, test payment marked PAID, cart emptied, stock decremented
Stock race	3	first buyer gets the last units, second buyer receives 409, stock is 0 and never negative
Order tracking	11	vendor sees only own items, cannot skip a step, CONFIRMED to DELIVERED, order status follows slowest item, timeline recorded
Cancellation	6	cancelled item returns stock, shipped item cannot be cancelled, second cancel refused
Invoice and statistics	5	invoice is a PDF, other customer cannot download it, vendor statistics exclude cancelled items, 7-day series present
Gateway and internal API	3	internal routes not exposed through the gateway, unknown route 404, catalog internal API rejects missing key 403

6.2 Worked pricing example

The cart in Figure 9 contains one Yoga Mat at Rs. 799 with coupon WELCOME10. The platform computed: discount 10% of 799 = Rs. 79.90, taxable amount = 719.10, GST 18% of 719.10 = Rs. 129.44, shipping Rs. 49 (the discounted amount is below Rs. 999), total = 719.10 + 129.44 + 49 = **Rs. 897.54**. The same figures appear on the order page (Figure 11) and on the invoice (Figure 12), which shows that the price is calculated once on the server and reused everywhere.

6.3 What the vendor dashboards show

After the demo data and my own test order were loaded, the four vendor dashboards (Figures 13, 15 and 16) reported the following. The numbers come straight from the screenshots and add up to the sample orders.

Vendor	Revenue	Orders	Units sold	Order status mix
TechHub	Rs. 4,999	1	1	1 delivered
Fashionista	Rs. 3,997	2	3	1 delivered, 1 shipped
HomeNest	Rs. 2,098	2	2	1 confirmed, 1 placed
PageTurner	Rs. 2,097	2	3	2 placed, 1 shipped

One customer order (#3) contains items from HomeNest and PageTurner, and both vendors see only their own line of it, which is the behaviour a marketplace needs.

6.4 Analysis

- **Consistency across services.** Checkout touches two databases (catalog for stock, orders for the order). Reserving stock first and releasing it on any later failure keeps the two consistent without a distributed transaction; the test 'failed checkout leaves stock untouched' proves it.
- **Concurrency.** Because the reservation is one atomic transaction, simultaneous buyers cannot oversell. This is the most important correctness property of any shop and is covered by an explicit test.
- **Per-item tracking.** In a multi-vendor order each vendor ships independently, so one order can be partly delivered. Tracking per item (with the order status derived from the items) models this correctly and gives vendors control of only their own goods.
- **Trusted pricing.** The browser sends only product ids and quantities. Prices, discounts, GST and totals are always computed on the server, so a customer cannot change the price by editing the request.
- **Fault isolation.** If the catalog service is down, the order service answers with HTTP 503 for the affected actions while the rest of the site remains available, and the launcher restarts a crashed service after two seconds.
- **Money handling.** Using integer paise avoids floating-point rounding errors; amounts are converted to rupees only for display.

6.5 Limitations

- Payments are simulated (test card and test UPI); no real payment gateway is connected, so no real money moves.
- SQLite is ideal for a demonstration; a production system would use PostgreSQL or MySQL, a message queue and containers.
- Product photos are vector illustrations; there is no image upload (a vendor can paste an image URL).
- There is no e-mail or SMS notification, refund workflow, product review system or administrator role.

7. Conclusion

The project delivers a complete working marketplace. Customers can discover products from several vendors, filter and sort them, build a cart, check out and track every item until delivery, and obtain a PDF invoice. Vendors can manage their catalogue, watch their sales and move their orders through fulfilment. The microservices architecture, with a database per service, a gateway and protected internal calls, meets the brief's aim of a production-style commerce architecture, and the 80 automated tests give confidence that the business rules hold.

The work gave practical experience with service decomposition, JWT authentication and role-based authorisation, transactional stock handling, pricing and tax calculation, PDF generation and building a responsive single-page interface without a framework.

Future enhancements: a real payment gateway, e-mail and SMS notifications, product image upload, reviews and ratings, an administrator console, a message queue between services, PostgreSQL, Docker containers and a CI pipeline.

8. References

- Node.js Documentation (fetch, node:sqlite, crypto) - https://nodejs.org/docs/latest/api/
- Express.js Guide - https://expressjs.com/ ; SQLite Documentation (WAL, transactions) - https://www.sqlite.org/docs.html
- JSON Web Token (RFC 7519) - https://www.rfc-editor.org/rfc/rfc7519 ; jsonwebtoken - https://github.com/auth0/node-jsonwebtoken
- PDFKit Documentation - https://pdfkit.org/
- S. Newman, *Building Microservices*, O'Reilly Media - patterns for database-per-service and API gateways
- Microservices pattern catalogue (API Gateway, Database per Service) - https://microservices.io/patterns/
- MDN Web Docs (Fetch API, Web Storage, SVG) - https://developer.mozilla.org/
- OWASP Top 10 - https://owasp.org/www-project-top-ten/